

Limitless Executive AI Coach

Building a client against the API

Everything is HTTP. There is no SDK and you do not need one. Thirty routes, grouped below by what you would build with them rather than by name.

Access via

```
https://limitless-executive.160-153-179-249.sslip.io
```

<https://limitless-executive.160-153-179-249.sslip.io/docs> is a live, interactive copy of this document. Every endpoint can be called from that page, which makes it the fastest way to see a real response shape.

CONTENTS

1	Authentication and conventions
2	The flow
3	Users and consent
4	Onboarding status
5	The cognitive assessment
6	Questionnaires
7	Check-ins
8	Reading the results
9	Coach and journal
10	Recommendations
11	Rendering a score correctly
12	Safety responses
13	Errors
14	Health probes

1. Authentication and conventions

The server is key-gated. Send the key as a header on every call except the two health probes:

```
X-API-Key: <the key>
```

Get the key from whoever deployed the server. It is not in the repository.

Server to server only. `user_id` is a request parameter, not an identity — anyone holding the key can pass any user id and read that person's data. Call this API from your backend, with your own session already verified. Calling it from the browser would hand every user a master credential.

Conventions

- JSON in, JSON out. Set **Content-Type: application/json** on writes.
- **user_id** goes in the query string on almost every call.
- Times are ISO 8601 with an offset. In a query string a `+` must be percent-encoded as **%2B**, or it arrives as a space and the request fails.
- Read endpoints accept an optional **as_of** parameter, which computes the answer as of that moment rather than now.

2. The flow

Three surfaces are what you are ultimately building. Everything else feeds them.

SURFACE	ENDPOINT	FASTEST ROUTE TO FILLING IT
Dashboard	GET <code>/v1/overview</code>	the 28-question self-assessment
Cognitive Health	GET <code>/v1/agents/cognitive-health</code>	the 9-minute task battery
Stress & Burnout	GET <code>/v1/agents/stress-burnout</code>	self-assessment plus the CBI

```
POST /v1/users -> user_id
GET /v1/onboarding/status -> what is still missing
POST /v1/self-assessment -> stress + energy, provisional cognition
POST /v1/assessment/session -> task sequence
POST /v1/assessment/trial x100 -> one per trial
POST /v1/assessment/complete -> measured brain_health + focus
POST /v1/instrument/cbi -> burnout
POST /v1/checkin -> keeps it current
GET /v1/overview -> the dashboard
```

Do not hardcode that sequence. GET `/v1/onboarding/status` reports what is missing and what each step unlocks, so when a score gains a component your client does not need redeploying to match.

3. Users and consent

POST `/v1/users`

Create a user and record their consents.

```
{
  "user_id": "your-own-id", # optional, generated if omitted
  "primary_timezone": "Asia/Kolkata", # required, IANA name
  "birth_year": 1985, # optional
  "education_years": 16, # optional
  "consents": ["core_service", "ai_coaching", "journal_analysis"]
}
```

```
{ "user": { "user_id": "...", "created_at": "...", "status": "active" },
  "created": true,
  "consents": [ ... ],
  "onboarding": { ... } }
```

Pass your own user id. Keeping it equal to your system's id means you never maintain a mapping table. Posting the same id twice returns the existing user with `"created": false`, so a retry after a dropped response is safe.

Birth year and education years sharpen the age- and education-adjusted normative comparison. They are optional on purpose: refusing to onboard someone who will not give their age is worse than a wider confidence band.

GET `/v1/users/{user_id}`

Profile, consent state and onboarding progress in one call.

PATCH `/v1/users/{user_id}`

Correct timezone, birth year or education years.

```
{ "primary_timezone": "Europe/London", "birth_year": 1978,
  "education_years": 20 }
```

All three optional; send what changed. Age and education feed the norm adjustment, so a correction moves every *future* cognitive score. Scores already written are untouched — the response says so.

GET `/v1/users/{user_id}/consent`

Current consent state, optionally as of a past date.

POST `/v1/users/{user_id}/consent`

Grant or revoke one purpose.

```
{ "purpose": "ai_coaching", "granted": false, "source": "settings" }
```

Grants and revocations are both appended, never updated in place, so “was this user consented on the 4th?” stays answerable. A revocation takes effect immediately, including for anything already cached.

GET

/v1/consent/purposes

Every purpose with its user-facing copy. Render your consent screen from this.

PURPOSE	NEEDED FOR
core_service	Everything. Without it every read returns 403
ai_coaching	POST /v1/coach/turn
journal_analysis	Journal entries and check-in notes
research_aggregate	De-identified model improvement
wearable_sync	Connected-device data, a later phase

Omit the consents field at signup and only core_service is granted; everything else is a separate opt-in.

DELETE

/v1/users/{user_id}

Delete the user and everything derived from them.

309 — the receipt

```
{ "status": "complete", "verified": true, "residue": {},  
  "total_rows_removed": 137, "blobs_removed": 2, "keys_destroyed": 1 }
```

Cascades across events, scores, decisions, cached features, journal blobs and their encryption keys, then scans every user-scoped table and reports what it found. **Keep the receipt** — it is what makes the deletion auditable. The identifier is not reusable afterwards; see 409.

4. Onboarding status

GET

`/v1/onboarding/status`

What the user still has to do, what each step unlocks, and why each score is empty.

```
{
  "complete": false,
  "next_step": "assessment",
  "estimated_seconds_remaining": 960,
  "steps": [ { "id": "assessment", "label": "Cognitive assessment",
  "detail": "Five timed tasks...", "done": false,
  "required": true,
  "unlocks": ["brain_health", "focus"],
  "endpoint": "POST /v1/assessment/session",
  "estimated_seconds": 540 } ],
  "features": {
    "cognitive_health": { "ready": false,
    "missing_any_of": [{"assessment", "self_assessment"} ] },
    "scores": {
      "energy": { "available": false, "provisional": false,
      "needs": "complete the 28-question self-assessment, or..." } }
  }
}
```

missing_any_of groups *alternatives* — any one member satisfies the group. That is how you offer the seven-minute questionnaire instead of routing everyone through the nine-minute battery.

unlocks lets you explain why you are asking rather than presenting a list of chores. **needs** is written to be shown to the user verbatim in an empty tile.

5. The cognitive assessment

Three calls: open a session, post each trial, close it.

POST

/v1/assessment/session

Plans the session. Returns the task sequence and stimulus forms.

```
{ "device_id": "...", "device_model": "Pixel 8", "os_version": "14",
  "timezone": "Asia/Kolkata" }
```

```
{ "assessment_id": "asm...",
  "task_sequence": ["trail_making_b", "corsi_span", "dual_n_back",
    "stroop", "word_list_delayed"],
  "forms": { "trail_making_b": "form_a", ... },
  "estimated_duration_s": 540,
  "requires_calibration": true,
  "excluded_tasks": { ... } }
```

Render exactly what is returned. Tasks in the order given, using the form id given. The order and form rotation are what keep repeat sessions comparable — shuffling them makes the practice-effect correction wrong. `excluded_tasks` is informational, not an error.

POST

/v1/assessment/trial

One trial result. Called about twenty times per task.

```
{ "assessment_id": "asm...", "task_id": "stroop", "trial_index": 7,
  "stimulus": { "word": "BLUE", "ink": "red" }, # object, not a string
  "response": { "choice": "red" }, # object, not a string
  "correct": true,
  "rt_ms_raw": 812.4, "rt_ms_corrected": 795.0,
  "form_id": "form_a", "interrupted": false,
  "device_id": "...", "timezone": "Asia/Kolkata",
  "idempotency_key": "asm...:stroop:7" }
```

stimulus and **response** are objects. Sending strings returns a 400 naming the field. Put whatever your task engine knows in them.

idempotency_key must be unique per trial and stable across retries; assessment id, task id and trial index concatenated works. A retried trial is then stored once — a key that varies stores it twice and biases the session.

POST

/v1/assessment/complete

Closes the session and scores it.

```
{ "assessment_id": "asm...", "task_ids": ["trail_making_b", ...],
  "completion_status": "complete", # or "partial" | "abandoned"
  "interruptions": 0, "timezone": "Asia/Kolkata" }
```

```
{ "accepted": true,
  "scores": [ { "name": "brain_health", "value": 68.6, "version": "1.0" } ] }
```

This call is not optional. A session you never close is not a completed assessment. The count it advances drives the practice-effect correction and the three-session personal baseline — without it the user's scores never get the practice term removed and their baseline never establishes. Call it even when the user quits; that is what *partial* and *abandoned* are for.

6. Questionnaires

GET `/v1/instrument/{name}`

The items to render, and whether it is due.

```
{ "instrument": "pss10", "due": true, "reason": "...",
  "items": [ { "item_id": "pss10_1", "text": "...",
    "min": 0, "max": 4, "reverse_scored": false } ] }
```

Render from the items. Do not hardcode the wording — scoring is keyed to these exact items and versions.

NAME	ITEMS	FEEDS
pss10	10	The stress score. Its largest component
cbi	13	Burnout. The only source of a burnout reading
who5	5	The well-being view

POST `/v1/instrument/{name}`

Submit answers. Returns the score and what it moved.

```
{ "item_responses": { "pss10_1": 2, ... }, "timezone": "...",
  "idempotency_key": "..." }
```

```
{ "instrument": "pss10", "value": 20.0, "completion": "complete",
  "scores_updated": [ { "name": "stress", "value": 47.1,
    "version": "1.1" } ] }
```

Use **scores_updated** to show the new number immediately rather than asking the user to refresh.

POST `/v1/self-assessment`

The 28-item Limitless self-assessment. The fastest way to fill the dashboard.

```
{ "item_responses": { "S1_Q1": 2, ... }, # preferred: the raw answers
  "elapsed_seconds": 430,
  "timezone": "Asia/Kolkata" }
```

```
# or, if you only have the scored output of the existing product:
{ "analyze_response": { "domains": {
  "attentionFocus": { "score": 62.5 }, ... } } }
```

Send the raw answers where you have them. A posted total cannot be re-scored if a model changes, and the response-validity checks need the item level. A scored result is accepted and marked for review.

Only the latest submission is used. Post history if you like — it is stored — but nothing reads across it.

SCORE	WHAT ONE QUESTIONNAIRE DOES
stress	A real component of the score, weighted 0.20
energy	A real component of the score, weighted 0.17

SCORE	WHAT ONE QUESTIONNAIRE DOES
brain_health	Stands in until the battery is sat
focus	Stands in until the battery is sat
burnout	Nothing. It needs the CBI

7. Check-ins

POST /v1/checkin

Mood, strain, energy, context and an optional note.

```
{ "mood": 4, "stress": 5, "energy": 3, # all 1-7
  "factors": ["deadline"], "note": "long week",
  "timezone": "Asia/Kolkata", "region": "IN" }
```

A note goes through the safety filter — see section 12. Pass **region** as an ISO country code so crisis resources are local. The response carries **scores_updated** and an **affects** list explaining what this check-in changed.

POST /v1/checkin/{checkin_type}

mood or sleep on its own.

```
# checkin_type = sleep
{ "payload": { "bedtime": "...", "waketime": "...",
  "quality": 3, "awakenings": 1 },
  "timezone": "...", "idempotency_key": "..." }
```

Expected, not a bug. Without the 28-item questionnaire, *energy* needs two mood check-ins and three nights of sleep before it appears. It is a trailing seven-day mean, and a mean of one observation is not a mean. Show the *needs* string from the onboarding status in the empty tile.

8. Reading the results

GET `/v1/overview`

Everything the dashboard needs, in one call.

```
{ "as_of": "...",
  "scores": [
    { "name": "brain_health", "available": true, "value": 68.6,
      "confidence_low": 64.8, "confidence_high": 78.9,
      "score_version": "1.0", "source": "cognitive",
      "provisional": false,
      "within_person": "in line with your typical range" },
    { "name": "energy", "available": false, "reason": "..." } ],
  "burnout": { "measured": true, "value": 50.0, "band": "moderate",
    "note": "..." },
  "wellbeing": { "value": 55.6, "band": "strained",
    "components": [ ... ], "missing": ["energy"] } }
```

One call rather than four, deliberately: splitting it would let the panels disagree by a few seconds of drift. **wellbeing.components** lists every weight, so you can show the working.

GET `/v1/dashboard`

Scores with 30-day trends, streaks and today's priorities.

GET `/v1/scores/{name}/history`

Time series for one score, plus the personal baseline.

Served under a **single score version** — mixing two would draw a trend across a definition change. Returns 404 when there is no history yet, rather than an empty series.

GET `/v1/agents/cognitive-health`

Score, percentile, baseline change, self-report gap, weekly plan.

GET `/v1/agents/stress-burnout`

Stress index, measured burnout, drivers, recovery actions.

```
{ "inputs": [ { "name": "...", "available": true, "value": ...,
  "unit": "..." } ],
  "outputs": { ... },
  "drivers": [ { "factor": "Work after 20:00",
    "direction": "lowering",
    "evidence": "45% of your activity was after 20:00",
    "modifiable": true, "magnitude": "moderate" } ],
  "plan": [ { "action": "...", "because": "...",
    "target": "...", "minutes": 10 } ],
  "withheld": { "cognitive_age": "...the reason..." },
  "notes": [ ... ] }
```

withheld names outputs deliberately not produced, each with its reason. Do not render these as missing features — they are product decisions, and the reason text explains them.

POST

/v1/agents/cognitive-health

The same agent against edited inputs. A what-if.

POST

/v1/agents/stress-burnout

The same, for stress and burnout.

```
{ "overrides": { "sleep_duration_mean_7d": 7.5, "stress_proxy_7d": 3.0 } }
```

Nothing is saved. The response carries **what_if.applied** listing what was substituted, and any projected score is flagged as a projection.

One asymmetry to reflect in the interface. Overrides move *stress* and *energy*, which are self-report measures. They cannot move *brain health*, which is measured by timed tasks — and the response says so in *what_if.why*. A slider that appeared to move it would be lying to the user.

GET

/v1/agents/inputs

The editable inputs with units, min, max and step.

Build your sliders from this rather than hardcoding the bounds.

9. Coach and journal

POST /v1/coach/turn

One conversational turn. Requires ai_coaching consent.

```
{ "message": "How has my sleep been this week?",  
  "conversation_id": "...", # omit on the first turn  
  "turn_index": 0, "region": "IN" }
```

```
{ "text": "You logged sleep on 4 of the last 7 nights...",  
  "conversation_id": "...", "tools_called": ["get_sleep_stats"],  
  "degraded": false, "coaching_suppressed": false,  
  "escalation": null }
```

Coach turns are slow by nature — they wait on a language model. Budget a longer client timeout for this endpoint than for the rest.

Every number in the reply is checked against what the tools actually returned before it is sent, so the coach cannot quote a figure it was not given.

POST /v1/journal

A free-text entry. Encrypted at rest; requires journal_analysis consent.

```
{ "text": "...", "timezone": "...", "region": "IN" }
```

10. Recommendations

GET /v1/recommendations

What to suggest now, or why nothing is being suggested.

POST /v1/recommendations/{decision_id}/outcome

Record that the user acted, dismissed or snoozed.

```
{ "outcome": "acted", "followup_delta": 1.5 }
```

Send this every time. Outcomes are how the recommender learns which suggestions are worth making. An interface that shows suggestions but never reports what happened leaves it permanently blind.

11. Rendering a score correctly

Four rules. Each exists because breaking it makes the product dishonest rather than merely untidy.

1. Always check available

A score with no data returns `available: false` and a **reason** written for the user. Never substitute zero, a dash, or last week's value.

2. Always render the confidence band

A bare score cannot even be constructed server-side. Two scores three points apart, on a measure whose interval spans nine, are the same score — showing the difference without the interval invites someone to act on noise.

3. Read source and provisional as separate questions

FIELD	QUESTION	VALUES
<code>source</code>	What produced this number	"cognitive" = timed tasks "self_report" = they told us
<code>provisional</code>	Is this a stand-in for a score that does not exist yet	true / false

Stress and energy are self-report by design and never provisional — do not label them estimates. A brain-health reading is provisional only while no battery has been sat; it carries a **caveat** and a **basis** written for the user, a null score version, and a band about three times wider.

When provisional is true, show the caveat. The number is what someone said about their attention, not what a task measured. Rendering it bare turns a self-report into a claim about cognition. Do not normalise the wider band away either — its width is the point. A measurement always displaces a stand-in, never the reverse.

4. Percentiles stay percentiles

The product reports a normative **percentile band**, never a cognitive age — and the withheld block carries the reason. Render it as a percentile; never convert it into years. Likewise **burnout** is a measured CBI result rather than a prediction, and its own note says so.

A provisional reading returns `percentile_band.available: false`, because a percentile places measured performance against a reference group and there is no measurement yet.

12. Safety responses

Any endpoint taking free text — the journal, a coach turn, and a check-in with a note — screens it and may return an escalation. Pass **region** as an ISO country code so the resources are local.

The two surfaces differ, and this will catch you out. The escalation object on a coach turn has *no message field*. The words to show are in the top-level **text**. A client written against the journal shape renders nothing on the coach.

	/V1/JOURNAL, /V1/CHECKIN	/V1/COACH/TURN
Wrapper	<code>safety</code>	<code>top_level</code>
Was it flagged	<code>safety.flagged</code>	<code>escalation is non-null</code>
Coach stayed quiet	<code>safety.coaching_suppressed</code>	<code>coaching_suppressed</code>
Words to show	<code>safety.message</code>	<code>text</code>
Resources	<code>safety.resources</code>	<code>escalation.resources</code>

When it fires, **show the words and the resources**. Do not suppress them, collapse them behind a tap, or restyle them into something dismissible. A suppressed coaching turn means the coach deliberately did not answer — rendering an empty reply instead of the escalation is the one failure mode that matters here.

13. Errors

CODE	MEANING	WHAT TO DO
400	Validation. The body names the field and the problem	Fix the payload; the message is specific
401	Missing or wrong X-API-Key	Configuration, not user error
403	Consent denied: the purpose is not granted	Send the user to the consent screen
404	No such user, or no history for that score	404 on history is normal early on
409	The user id belonged to a deleted user	Deleted ids are never reused. Pick another
503	/v1/ready only: database unreachable	Retry, and page someone

```
400: {"detail": {"reason": "payload_invalid",
"detail": {"errors": [ { "field": "stimulus",
"problem": "Input should be a valid dictionary" } ] } } }
```

Two ingestion rules that surprise people

- **Events more than a day ahead of server time are refused** as clock skew. Do not send optimistic future timestamps.
- **Events dated before the user's consent grant are refused.** You cannot backfill history from before signup.

14. Health probes

GET

/v1/health

Liveness. The process is running. No API key needed.

GET

/v1/ready

Readiness. Actually reaches the database. No API key needed.

```
{ "status": "ready", "checks": { "database": "ok" } }
```

Point monitoring at **/v1/ready**, not **/v1/health**. A process that cannot reach its database serves 500s while reporting itself perfectly healthy.

Live interactive reference: <https://limitless-executive.160-153-179-249.sslip.io/docs>